

arquitectura completa (DB + Edge + UI + dedupe + riesgo) para el flujo “Control de Reservas / Importación de Llegadas” y manteniendo RGPD razonable.

1) Concepto del módulo

Nombre funcional: **Control de Reservas (Pre-Checkin Screening)**

Dos modos:

1. **Carga inicial** (una vez): huéspedes actuales / histórico reciente (opcional)
2. **Control diario**: próximos check-in (día a día o semana)

Objetivo UI tras subir CSV:

- Por cada registro importado mostrar:
COINCIDENCIA / NO COINCIDENCIA / RIESGO (ALTO-MEDIO-BAJO)
 - nº coincidencias y resumen (sin PII global, solo si procede)

Y en BD:

- Guardas un **perfil neutral** (sin rating) + **eventos de estancia/import.**
 - Si luego hay incidente, se crea **evaluation** real y se recalcula riesgo.
-

2) Modelo de datos recomendado (mínimo sólido)

2.1 `guest_profiles` (perfil neutral por hotel)

Una fila por “persona” **dentro de un hotel (org_id)**.

Campos clave:

- `id` `uuid` `pk`
- `org_id` `uuid` `not null`
- `guest_key` `text` `not null` → tu fingerprint estable (ver punto 3)
- `full_name` `text` `null` (normalizado)
- `nationality_alpha3` `text` `null`
- `email_norm` `text` `null` (lower)
- `phone_norm` `text` `null` (E164 o dígitos)
- `doc_norm` `text` `null` (upper alfanum)
- métricas:
 - `stays_count` `int` `not null` `default 0`

- last_seen_at timestamptz null
- first_seen_at timestamptz null
- incidents_count int not null default 0
- total_gross numeric not null default 0
- total_recovered numeric not null default 0
- total_net_loss numeric not null default 0
- estado/etiqueta:
 - status text not null default 'NO_INCIDENTS' (NO_INCIDENTS | HAS_INCIDENTS | WATCHLIST)
 - internal_label text null (NEUTRO/VIP/OBSERVAR...)
- auditoría:
 - created_at, updated_at

ÚNICO: (org_id, guest_key).

Importante: este perfil **no tiene rating**.

2.2 guest_stays (evento de estancia/import)

Cada “aparición” (checkin planificado o estancia real).

- id uuid pk
- org_id uuid not null
- guest_profile_id uuid not null
- source text not null (CSV | PMS | MANUAL)
- source_batch_id uuid null (para agrupar importación)
- checkin_date date null
- checkout_date date null
- reservation_ref text null (si viene)
- created_at timestamptz not null

Índices:

- (org_id, checkin_date)
- (guest_profile_id, checkin_date desc)

2.3 import_batches

Para trazabilidad y repetir cargas sin duplicar.

- `id` uuid pk
 - `org_id` uuid not null
 - `source` text not null (CSV/PMS)
 - `file_name` text null
 - `rows_total` int not null
 - `rows_new_profiles` int not null
 - `rows_new_stays` int not null
 - `rows_duplicates` int not null
 - `created_at` timestampz not null
 - `created_by` uuid not null (auth.user)
-

2.4 import_rows (opcional pero MUY útil)

Para mostrar resultados y depurar. Puedes guardar 7–30 días y purgar.

- `id` uuid pk
 - `batch_id` uuid not null
 - `row_index` int not null
 - `raw` jsonb not null (fila original)
 - `normalized` jsonb not null (doc/email/phone/nationality/full_name)
 - `match_status` text not null (EXACT | POSSIBLE | NONE)
 - `risk_level` text not null (LOW | MEDIUM | HIGH)
 - `matches` jsonb null (lista de coincidencias con ids + resumen)
 - `created_at` timestampz not null
-

2.5 evaluations (tu sistema actual)

No lo toques conceptualmente:

- solo se crean cuando el hotel **hace una valoración** o registra incidente.
- aquí es donde existe `rating`, `incident_type`, `economic_*`, `evaluation_date`, etc.

3) Dedupe y “guest_key” (lo crítico)

Tú quieres evitar insertar repetidos al recargar.

Normalización

- `doc_norm`: alfanum uppercase sin símbolos.
- `email_norm`: trim + lower.
- `phone_norm`: dígitos o E164 (recomiendo E164 si puedes; si no, dígitos y ya).
- `nationality_alpha3`: solo [A-Z]{3} (select).
- `full_name_norm`: uppercase, sin números, colapsa espacios.

Jerarquía de clave

1. Si hay documento: `guest_key = "DOC:" + doc_norm`
2. Si no doc pero hay email: `guest_key = "EMAIL:" + email_norm`
3. Si no doc/email pero hay phone: `guest_key = "PHONE:" + phone_norm`
4. Si no hay ninguno → **no crees perfil** (o lo creas “TEMP:hash(nombre+fecha)” pero es mala idea).

Esto te da **idempotencia**: mismo huésped → mismo `guest_key` → UPSERT.

4) Política RGPD práctica (sin posturo)

- Perfil neutral: “**SIN INCIDENCIAS REGISTRADAS**” es un hecho operativo.
 - No muestres PII global a otros hoteles.
 - En consultas globales: solo **señales agregadas** (coincide/no coincide + nº incidentes + rango temporal), y si quieres, “severidad máxima observada”.
 - Mantén `import_rows` con TTL (purgado).
-

5) Cálculo de riesgo (simple y útil)

No dependas del rating 5.

Para screening de llegadas, define `risk_level` por regla:

Si hay coincidencia con incidentes previos (en el mismo hotel):

- `incidents_count = 0` → LOW

- 1 incidente leve (severity 1–2) y hace > 12 meses → MEDIUM
- reincidente (>=2) o severidad >=4 o net_loss alto → HIGH

Si hay coincidencia global (otros hoteles):

- Nunca muestras detalles, solo:
 - `global_signal = true`
 - `global_incidents_bucket = 1 | 2-3 | 4+`
 - `global_last_seen_bucket = <6m | 6-24m | >24m`
- `risk_level`:
 - 1 y >24m → MEDIUM
 - 2-3 o <24m → HIGH
 - 4+ → HIGH

Si no hay coincidencia:

- LOW

Esto ya sirve para el hotel: “vigila” vs “normal”.

6) Flujo Edge Functions (sin inventar magia)

6.1 `debacu_eval_import_validate` (opcional)

- recibe el CSV parseado (o primero subes archivo a storage y luego lo lees)
- devuelve:
 - columnas detectadas
 - errores por fila
 - preview 20 filas

6.2 `debacu_eval_import_commit`

Input:

- `org_id` (del JWT, no del cliente)
- `mode`: INITIAL | DAILY
- `rows`: array normalizado
(doc/email/phone/name/nationality/checkin/checkout/reservation_ref)

Proceso:

1. crea `import_batches`

2. por cada fila:

- normaliza
- calcula `guest_key`
- busca exact match (`org_id`, `guest_key`)
- upsert en `guest_profiles`
- inserta `guest_stays` si `checkin_date` no duplicado:
 - regla anti duplicado: `unique (guest_profile_id, checkin_date, source)` o dedupe manual
- calcula `match_status` y `risk_level` (usando:
 - incidentes del mismo hotel
 - señal global si existe)
- guarda en `import_rows` (opcional)

3. devuelve resumen + lista paginada de resultados

6.3 `debacu_eval_import_rows_list`

- lista resultados por `batch_id` (paginado)

6.4 `debacu_eval_guest_profile_get`

- devuelve:
 - ficha neutral (`stays_count`, `last_seen`, `incidents_count`, totales)
 - timeline de stays
 - timeline de incidents (evaluations)
-

7) UI que te recomiendo (operativo de verdad)

Pantalla “Control de Reservas”

Bloques:

1. Subir CSV

- plantilla descargable (cabeceras)
- mapping columnas (si no coincide)
- selector “modo”: Inicial / Diario

2. Resultados del batch

Tabla con:

- Cliente (nombre si viene)
- Identificador usado (DOC/EMAIL/PHONE – parcialmente enmascarado)
- Nacionalidad (select)
- Checkin date
- Estado: COINCIDENCIA / NO COINCIDENCIA
- Riesgo: LOW/MEDIUM/HIGH (badge)
- Incidencias: 0 / 1 / 2-3 / 4+ (bucket)
- Acción: “Ver ficha” (drawer)

3. Drawer “Ficha del cliente”

- SIN INCIDENCIAS REGISTRADAS ☒ (si incidents_count=0)
- Estancias registradas: X
- Última estancia: fecha
- Incidencias: 0
- Pérdidas/recuperado/neto: 0/0/0
- Botón: “Registrar incidencia” (lleva al RatingForm pre-relleno)

8) Lo que NO haría

- No guardaría “5 estrellas” por defecto.
- No guardaría perfiles si no hay doc/email/phone (te llena de basura).
- No confiaría en nombre + país como clave (colisiones brutales).
- No mostraría detalles globales (te metes en lío).

DEBACU EVALUATION

Documento Base – Arquitectura Importaciones + Inteligencia Global

1 Objetivo Estratégico

Convertir Debacu Evaluation en:

- ☒ Sistema de inteligencia colectiva entre hoteles
 - ☒ Motor de control de riesgo en pre-checkin
 - ☒ Plataforma operativa para revenue / forecasting / no-show
 - ☒ Base estadística histórica por hotel
 - ☒ Sin romper lo ya listo para producción
-

2 Principios Clave (No negociables)

1. ☒ No romper debacu_evaluations actual.
 2. ☒ No meter rating=5 artificial.
 3. ☒ No exponer qué hotel registró qué dato.
 4. ☒ No almacenar PII en la capa global.
 5. ☒ Separar claramente:
 - Global intelligence
 - Datos operativos por hotel
 - Revenue / Forecast
 6. ☒ Todo en snake_case.
 7. ☒ Nuevas tablas con prefijo debacu_eval_import_*
-

3 Modelo Conceptual Definitivo

CAPA GLOBAL (Inteligencia colectiva)

Tabla actual

debacu_evaluations

- Sigue existiendo.
- Contiene incidencias / valoraciones reales.
- Es la fuente de señales negativas.

Nueva tabla global (sin PII)

debacu_eval_import_guest_index

Contendrá:

- identity_key (hash estable HMAC)
- stays_count
- reservations_count
- incidents_count
- total_gross
- total_recovered
- total_net_loss
- first_seen_date
- last_seen_date
- risk_band (LOW/MEDIUM/HIGH)

⚠ Aquí NO hay documento, email ni teléfono en claro.

CAPA HOTEL (Operativa privada por org)

Tablas nuevas:

debacu_eval_import_batches

Registro de cada carga CSV/PMS.

debacu_eval_import_rows

Resultado por fila:

- match_status
- risk_band_at_import
- identity_key
- checkin_date
- checkout_date
- error_code

debacu_eval_guest_stays

Reservas y estancias reales:

- stay_status:

- PLANNED
- CHECKED_IN
- COMPLETED
- NO_SHOW
- CANCELLED

Esta tabla alimenta:

- Forecast
- Estadísticas
- No-show ratio
- Historial interno

4 Flujo Operativo Definitivo

A) Importación CSV (Control de Reservas)

1. Usuario sube CSV.
2. Edge normaliza datos.
3. Se calcula identity_key (HMAC).
4. Se consulta capa global.
5. Se devuelve:
 - Coincidencia global: Sí/No
 - SIN INCIDENCIAS REGISTRADAS ☒
 - O RIESGO MEDIO/ALTO
 - N° señales
 - Última actividad
6. Se guardan:
 - import_batch
 - import_rows
 - guest_stays (PLANNED o COMPLETED)
7. Se actualiza guest_index global.

B) Cuando hay incidencia (manual)

1. Se crea registro en debacu_evaluations
 2. Se actualiza:
 - incidents_count en guest_index
 - totales económicos
 - risk_band recalculado
-

C) No-show

- stay_status = NO_SHOW
 - Se crea evaluation tipo NO_SHOW
 - Se actualiza índice global
-

5) Cómo se mostrará en Consulta GLOBAL

El hotel que consulta verá:

Si no hay incidencias:

SIN INCIDENCIAS REGISTRADAS ☒

Estancias registradas: X

Última actividad: fecha

Incidencias: 0

Pérdidas acumuladas: 0

Si hay incidencias:

COINCIDENCIA GLOBAL 

Nivel de riesgo: MEDIO / ALTO

Incidencias registradas: N

Severidad máxima observada: X

Última incidencia: fecha

Pérdidas acumuladas: XX €

Nunca se mostrará:

- Qué hotel lo reportó
 - Qué comentario
 - Datos personales completos
-

6 Diferencia Clave

Capa	Contiene	Finalidad
debacu_evaluations	Incidencias reales	Señales negativas globales
debacu_eval_import_*	Operativa por hotel	Revenue / Forecast / Control
guest_index global	Agregados anónimos	Consulta global

7 Política sobre “Buenos”

NO se crea rating 5 por defecto.

Se considerará “cliente limpio” cuando:

- incidents_count = 0
- stays_count >= 1

Y se mostrará como:

“SIN INCIDENCIAS REGISTRADAS”

Eso es un hecho, no una opinión.

8 Dedupe y Reglas

- Identidad global por HMAC de doc/email/phone.
 - Dedupe de estancias por:
hotel_id + identity_key + checkin_date
 - Un cliente puede tener múltiples fechas distintas.
 - No se repite la misma reserva si se reimporta el mismo fichero.
-

9 Riesgo (Modelo Inicial)

risk_band:

LOW:

- incidents_count = 0

MEDIUM:

- 1 incidente leve o antiguo

HIGH:

- reincidencia

- severidad alta
- pérdidas económicas relevantes

(Se podrá sofisticar después.)

10 Fase de Implementación

Orden correcto:

1. Crear tablas nuevas.
 2. Crear guest_index global.
 3. Implementar import Edge Function.
 4. Implementar consulta global unificada.
 5. Añadir stay_status y no-show.
 6. Conectar revenue/forecast.
-

🔒 Decisión Arquitectónica Final

- debacu_evaluations NO se elimina.
 - No se sustituye.
 - No se llena de “buenos”.
 - Se mantiene como núcleo de señales negativas.
 - El sistema crece alrededor.
-

🎯 Resultado Final del Producto

Debacu será:

- ✓ Inteligencia colectiva real
- ✓ Sin exposición de identidad
- ✓ Con histórico operativo por hotel
- ✓ Con motor revenue independiente
- ✓ Escalable
- ✓ Defendible legalmente

FASE 1 — Capa Global Anónima

Tabla: debacu_eval_import_guest_index

Esta tabla será el **índice global anónimo** que:

- Permite match entre hoteles
 - No guarda PII
 - Alimenta la consulta GLOBAL
 - Se actualiza con importaciones y con incidencias
-

1 Diseño Final de la Tabla

```
create table if not exists debacu_eval_import_guest_index (  
  id uuid primary key default gen_random_uuid(),  
  -- identidad anónima (HMAC estable)  
  identity_key text not null unique,  
  -- métricas globales  
  stays_count integer not null default 0,  
  reservations_count integer not null default 0,  
  incidents_count integer not null default 0,  
  total_gross numeric not null default 0,  
  total_recovered numeric not null default 0,  
  total_net_loss numeric not null default 0,  
  first_seen_date date null,  
  last_seen_date date null,  
  last_incident_date date null,  
  risk_band text not null default 'LOW',  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now(),  
  constraint debacu_eval_import_guest_index_risk_chk  
    check (risk_band in ('LOW','MEDIUM','HIGH'))  
);
```

2 Índices recomendados

```
create index if not exists debacu_eval_import_guest_index_risk_idx  
  on debacu_eval_import_guest_index (risk_band);  
create index if not exists debacu_eval_import_guest_index_last_seen_idx  
  on debacu_eval_import_guest_index (last_seen_date desc);  
create index if not exists debacu_eval_import_guest_index_last_incident_idx  
  on debacu_eval_import_guest_index (last_incident_date desc);
```

3 Trigger updated_at automático

```
create or replace function debacu_eval_set_updated_at()  
returns trigger language plpgsql as $$  
begin
```

```
new.updated_at = now();
return new;
end $$;
drop trigger if exists debacu_eval_import_guest_index_set_updated_at
on debacu_eval_import_guest_index;
create trigger debacu_eval_import_guest_index_set_updated_at
before update on debacu_eval_import_guest_index
for each row execute function debacu_eval_set_updated_at();
```

4 RLS (Muy importante)

Como esto es GLOBAL, pero solo accesible vía Edge:

```
alter table debacu_eval_import_guest_index enable row level security;
-- Solo service_role debe acceder directamente.
-- No creamos políticas públicas.
```

⚠ El acceso será solo mediante Edge Function con service_role.

Los hoteles nunca consultan directamente esta tabla.

FASE 2 — identity_key (lo crítico)

Esto NO se hace en SQL.

Se hace en Edge.

Modelo definitivo:

```
identity_key = HMAC_SHA256(
  GLOBAL_SECRET_PEPPER,
  normalized_identifier
)
```

Jerarquía:

1. Documento
2. Email
3. Teléfono

Ejemplo conceptual:

DOC:11792515R
EMAIL:fercarboc@gmail.com
PHONE:34660082474
Luego HMAC.

Esto garantiza:

- Match cross-hotel

- No reversibilidad
- No exposición PII
- Estabilidad en el tiempo

FASE 3 — Actualización del índice global

A) Cuando importas estancia sin incidencia

```
stays_count++  
last_seen_date = max(last_seen_date, checkin_date)  
if first_seen_date is null -> set  
risk_band recalculado (seguirá LOW si no hay incidencias)
```

B) Cuando hay incidencia

```
incidents_count++  
total_gross += x  
total_recovered += y  
total_net_loss += z  
last_incident_date = evaluation_date  
recalcular risk_band
```

◇ Reglas iniciales de risk_band

Muy simple al principio:

```
if incidents_count = 0 -> LOW  
if incidents_count = 1 and total_net_loss < 200 -> MEDIUM  
if incidents_count >= 2 or total_net_loss >= 200 -> HIGH  
Esto se podrá sofisticar después.
```

◇ Resultado para consulta GLOBAL

Edge Function devolverá algo así:

```
{  
  "exists": true,  
  "risk_band": "LOW",  
  "stays_count": 3,  
  "incidents_count": 0,  
  "last_activity": "2026-02-10",  
  "total_net_loss": 0  
}
```

Y en UI:

SIN INCIDENCIAS REGISTRADAS ☒

Estancias registradas: 3

Fase 1 sólida y sin tocar nada crítico.

Ahora vamos con la **Fase 2 — Infraestructura de Importación por Hotel**.

Aquí empezamos a construir el motor operativo real.

◇ FASE 2 — Capa Operativa por Hotel

Crearemos 3 tablas:

1. debacu_eval_import_batches
2. debacu_eval_import_rows
3. debacu_eval_guest_stays

Nada de esto afecta a debacu_evaluations.

1.1 Tabla: debacu_eval_import_batches

Controla cada carga CSV / PMS.

```
create table if not exists debacu_eval_import_batches (  
  id uuid primary key default gen_random_uuid(),  
  org_id uuid not null,  
  file_name text null,  
  import_type text not null, -- INITIAL_LOAD | DAILY_CHECKIN | HISTORICAL  
  total_rows integer not null default 0,  
  processed_rows integer not null default 0,  
  error_rows integer not null default 0,  
  created_at timestamptz not null default now(),  
  constraint debacu_eval_import_batches_type_chk  
    check (import_type in ('INITIAL_LOAD', 'DAILY_CHECKIN', 'HISTORICAL'))  
);
```

Índice

```
create index if not exists debacu_eval_import_batches_org_idx  
on debacu_eval_import_batches (org_id, created_at desc);
```

2.2 Tabla: debacu_eval_import_rows

Resultado por fila del CSV.

```
create table if not exists debacu_eval_import_rows (  
  id uuid primary key default gen_random_uuid(),
```

```

    batch_id uuid not null references debacu_eval_import_batches(id) on delete
cascade,
    org_id uuid not null,
    identity_key text not null,
    full_name text null,
    checkin_date date not null,
    checkout_date date null,
    match_status text not null default 'NO_MATCH',
    risk_band_at_import text null,
    error_code text null,
    created_at timestamptz not null default now(),
    constraint debacu_eval_import_rows_match_chk
        check (match_status in
('NO_MATCH', 'MATCH_LOW', 'MATCH_MEDIUM', 'MATCH_HIGH', 'ERROR'))
);

```

Índices

```

create index if not exists debacu_eval_import_rows_org_idx
on debacu_eval_import_rows (org_id);
create index if not exists debacu_eval_import_rows_identity_idx
on debacu_eval_import_rows (identity_key);
create index if not exists debacu_eval_import_rows_checkin_idx
on debacu_eval_import_rows (checkin_date desc);

```

3.3 Tabla: debacu_eval_guest_stays

Aquí guardamos reservas y estancias reales del hotel.

```

create table if not exists debacu_eval_guest_stays (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    identity_key text not null,
    full_name text null,
    checkin_date date not null,
    checkout_date date null,
    stay_status text not null default 'PLANNED',
    import_batch_id uuid null references debacu_eval_import_batches(id) on delete
set null,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    constraint debacu_eval_guest_stays_status_chk
        check (stay_status in
('PLANNED', 'CHECKED_IN', 'COMPLETED', 'NO_SHOW', 'CANCELLED'))
);

```

◇ Regla crítica de DEDUPE

Un huésped puede venir muchas veces, pero no repetir la misma estancia.

```

create unique index if not exists debacu_eval_guest_stays_unique_idx
on debacu_eval_guest_stays (org_id, identity_key, checkin_date);

```

Esto evita:

- Subir el mismo CSV dos veces
- Duplicar reservas

Pero permite:

- Mismo huésped en fechas distintas
-

◇ updated_at trigger

```
drop trigger if exists debacu_eval_guest_stays_set_updated_at
on debacu_eval_guest_stays;
create trigger debacu_eval_guest_stays_set_updated_at
before update on debacu_eval_guest_stays
for each row execute function debacu_eval_set_updated_at();
```

◇ RLS (muy importante)

Estas tablas son por hotel.

```
alter table debacu_eval_import_batches enable row level security;
alter table debacu_eval_import_rows enable row level security;
alter table debacu_eval_guest_stays enable row level security;
```

Después crearemos políticas por org_id.

◇ Estado tras Fase 2

Ahora tienes:

- ☒ Índice global anónimo
- ☒ Motor de importación por hotel
- ☒ Control de batches
- ☒ Dedupe seguro
- ☒ Base para forecasting
- ☒ Base para control de no-show

Y seguimos sin tocar debacu_evaluations.

Seguimos con fase 3

Ahora entramos en la **parte crítica de verdad**: cómo conectar todo sin romper producción.

Estamos en:

- ☒ Fase 1: guest_index global
 - ☒ Fase 2: tablas import hotel
 -  Fase 3: lógica que une todo
-

FASE 3 — Lógica de Actualización del Índice Global

Antes de hacer la Edge de importación, necesitamos **funciones SQL seguras** que:

1. Creen / actualicen `debacu_eval_import_guest_index`
2. Recalculen `risk_band`
3. Sean reutilizables tanto por:
 - Importaciones
 - Nuevas incidencias (`debacu_evaluations`)

Esto es clave para que el sistema sea consistente.

1.1 Función: `upsert_guest_index_from_stay`

Cuando se importa una estancia (sin incidencia).

```
create or replace function debacu_eval_upsert_guest_index_from_stay(  
    p_identity_key text,  
    p_activity_date date  
)  
returns void  
language plpgsql  
as $$  
begin  
    insert into debacu_eval_import_guest_index (  
        identity_key,  
        stays_count,  
        first_seen_date,  
        last_seen_date  
    )  
    values (  
        p_identity_key,  
        1,  
        p_activity_date,  
        p_activity_date  
    )  
    on conflict (identity_key)  
    do update set  
        stays_count = debacu_eval_import_guest_index.stays_count + 1,  
        last_seen_date = greatest(  
            debacu_eval_import_guest_index.last_seen_date,  
            p_activity_date  
        );  
end;  
$$;
```

2.2 Función: upsert_guest_index_from_incident

Cuando se crea una incidencia en debacu_evaluations.

```
create or replace function debacu_eval_upsert_guest_index_from_incident(
  p_identity_key text,
  p_evaluation_date date,
  p_gross numeric,
  p_recovered numeric,
  p_net_loss numeric
)
returns void
language plpgsql
as $$
begin
  insert into debacu_eval_import_guest_index (
    identity_key,
    incidents_count,
    total_gross,
    total_recovered,
    total_net_loss,
    first_seen_date,
    last_seen_date,
    last_incident_date
  )
  values (
    p_identity_key,
    1,
    coalesce(p_gross,0),
    coalesce(p_recovered,0),
    coalesce(p_net_loss,0),
    p_evaluation_date,
    p_evaluation_date,
    p_evaluation_date
  )
  on conflict (identity_key)
  do update set
    incidents_count = debacu_eval_import_guest_index.incidents_count + 1,
    total_gross = debacu_eval_import_guest_index.total_gross +
coalesce(p_gross,0),
    total_recovered = debacu_eval_import_guest_index.total_recovered +
coalesce(p_recovered,0),
    total_net_loss = debacu_eval_import_guest_index.total_net_loss +
coalesce(p_net_loss,0),
    last_incident_date = p_evaluation_date,
    last_seen_date = greatest(
      debacu_eval_import_guest_index.last_seen_date,
      p_evaluation_date
    );
  -- Recalcular risk_band
  update debacu_eval_import_guest_index
  set risk_band =
    case
      when incidents_count = 0 then 'LOW'
      when incidents_count = 1 and total_net_loss < 200 then 'MEDIUM'
      else 'HIGH'
    end
  where identity_key = p_identity_key;
end;
$$;
```

Importante

Estas funciones:

- ✗ No tocan debacu_evaluations
- ✗ No rompen nada existente
- ☒ Solo actualizan la nueva infraestructura
- ☒ Son reutilizables desde Edge

seguimos con fase 4

DEBACU_GLOBAL_PEPPER= b9d3a1f4e8c6d2a97f82d91ac8c14b8e9b5f44a7e20f91c7

PASO 2 — Rellenar identity_key retroactivamente

⚠ Esto debe hacerse desde backend o SQL con función HMAC.

Asumimos que usarás document_norm como prioridad, luego email_norm, luego phone_digits.

Ejemplo en SQL usando digest (si tienes pgcrypto):

```
update debacu_evaluations
set identity_key =
    encode(
        hmac(
            coalesce(document_norm, email_norm, phone_digits),
            'TU_PEPPER_GLOBAL_SECRETO',
            'sha256'
        ),
        'hex'
    )
where identity_key is null
and coalesce(document_norm, email_norm, phone_digits) is not null;
```

⚠ El pepper debe ser el mismo que usarás en Edge.

⚠ Mejor hacerlo desde una Edge script si prefieres no meter el secreto en SQL.

PASO 3 — Insertar datos agregados en guest_index

Ahora construimos el índice completo:

```
insert into debacu_eval_import_guest_index (  
    identity_key,  
    incidents_count,  
    total_gross,  
    total_recovered,  
    total_net_loss,  
    first_seen_date,  
    last_seen_date,  
    last_incident_date,  
    risk_band  
)  
select  
    identity_key,  
    count(*) as incidents_count,  
    coalesce(sum(economic_impact_gross),0),  
    coalesce(sum(economic_recovered),0),  
    coalesce(sum(economic_net_loss),0),  
    min(evaluation_date),  
    max(evaluation_date),  
    max(evaluation_date),  
    case  
        when count(*) = 0 then 'LOW'  
        when count(*) = 1 and coalesce(sum(economic_net_loss),0) < 200 then 'MEDIUM'  
        else 'HIGH'  
    end  
from debacu_evaluations  
where identity_key is not null  
group by identity_key  
on conflict (identity_key) do nothing;
```

PASO 4 — Verificación

Comprobar volumen:

```
select count(*) from debacu_eval_import_guest_index;
```

Debe ser menor o igual que el número de identidades únicas en debacu_evaluations.

Comprobar ejemplo:

```
select *  
from debacu_eval_import_guest_index  
order by incidents_count desc  
limit 10;
```

Resultado

Ahora:

- Todo tu histórico de 10.200 registros

- Está indexado globalmente
 - Con risk_band calculado
 - Con agregados económicos
 - Sin tocar lógica actual
-

MUY IMPORTANTE

A partir de ahora:

Cada vez que crees una nueva incidencia en `debacu_evaluations`, debes llamar automáticamente:

```
debacu_eval_upsert_guest_index_from_incident(...)
```

Si no lo haces, el índice se desincroniza.

Estado actual

- ✓ Histórico reconstruido
- ✓ Infraestructura import creada
- ✓ Índice global coherente

Vamos a analizarlo con frialdad:

Lo que está BIEN

1. `incidents_count` correcto (agregado por `identity_key`).
2. Totales económicos acumulados correctamente.
3. `first_seen_date`, `last_seen_date`, `last_incident_date` coherentes.
4. `risk_band` calculado según regla.
5. `stays_count` = 0 → lógico, aún no hemos importado estancias.

Hasta aquí, la reconstrucción está bien hecha.

Lo que debes entender ahora

Tu índice global ahora mismo refleja:

Solo historial de incidencias.

Todavía no refleja:

- Estancias buenas
- Reservas futuras
- Clientes sin incidentes

Eso llegará cuando empieces a importar huéspedes.

Observación importante sobre risk_band

Ahora mismo tienes:

- 1 incidente → MEDIUM
- 2 incidentes → HIGH
- 3+ → HIGH

Eso es agresivo.

Ejemplo:

```
incidents_count = 2  
total_net_loss = 7.50  
risk_band = HIGH
```

Eso puede ser demasiado duro.

Podemos suavizar luego, pero no es crítico ahora.

Interpretación estratégica

Lo que tienes ahora es:

Base global de señales negativas ya indexada.

Eso significa que:

Cuando el Hotel B consulte,
ya puede saber:

- Si ese cliente tiene historial negativo.
- Cuántas incidencias.
- Nivel de riesgo agregado.

Eso es exactamente lo que querías.



Estado real del sistema ahora

Capa	Estado
debaqu_evaluations	Producción estable
guest_index global	Reconstruido
import tables	Creadas
matching	Aún no implementado
estancias buenas	Aún no alimentadas

Qué cambia ahora conceptualmente

Hasta ahora:

guest_index = solo incidencias

A partir de ahora:

```
guest_index =
  stays_count
+
  incidents_count
+
  totales económicos
+
  risk_band
```

Y eso permite mostrar:

Caso 1 – Cliente limpio

- Estancias registradas: 5
- Incidencias: 0
- SIN INCIDENCIAS REGISTRADAS ☒

Caso 2 – Cliente con historial

- Estancias registradas: 4
- Incidencias: 2
- Nivel de riesgo: MEDIUM

Eso es justo lo que querías para el segundo hotel.



Lo que debemos ajustar antes de seguir

Tu función actual:

debacu_eval_upsert_guest_index_from_stay

Solo incrementa `stays_count`.

Pero ahora necesitamos que:

1. También mantenga `reservations_count`
2. Recalcule `risk_band` correctamente
3. No deje inconsistencias

Ajuste correcto de `risk_band` (modelo más equilibrado)

La regla actual es demasiado simple.

Propongo esta versión más lógica:

```
if incidents_count = 0 -> LOW
if incidents_count = 1 and total_net_loss < 200 -> MEDIUM
if incidents_count >= 2
  OR total_net_loss >= 300 -> HIGH
```

Así no marcas HIGH por 2 incidentes pequeños.

¿Cuándo se actualiza `stays_count`?

- reservation import → `reservations_count++`
- estancia completada → `stays_count++`

Porque si no, distorsionas forecasting.

Flujo definitivo que vamos a implementar

Cuando se importe CSV:

Para cada fila:

1. Generar `identity_key`
2. Insertar `guest_stays` (PLANNED)
3. Incrementar `reservations_count` en `guest_index`
4. Si es estancia histórica (COMPLETED):
 - Incrementar `stays_count`

5. Consultar índice para devolver riesgo

Resultado final esperado

En el índice global verás algo así:

identity_key	stays	reservations	incidents	risk
X1	5	6	0	LOW
X2	4	4	2	MEDIUM
X3	0	1	3	HIGH

Mostrar número de estancias (pero no qué hoteles).

Mostrar número de estancias globales sin revelar hoteles:

- ✓ Aporta contexto real.
- ✓ No viola anonimato.
- ✓ No rompe RGPD.
- ✓ Da valor operativo al segundo hotel.

Ahora vamos a cerrar el diseño definitivo antes de escribir la Edge.

Modelo final del índice global

Tu tabla `debacu_eval_import_guest_index` ahora será la fuente de verdad para consulta global.

Debe reflejar:

- `stays_count`
- `reservations_count`
- `incidents_count`
- `total_net_loss`
- `risk_band`
- `last_seen_date`

- `last_incident_date`

Y la UI mostrará:

Consulta Global — Respuesta Final

Si no hay incidencias:

SIN INCIDENCIAS REGISTRADAS ☒

Estancias registradas globalmente: 7

Última actividad: 2026-02-20

Si hay incidencias:

COINCIDENCIA GLOBAL 

Nivel de riesgo: MEDIUM

Estancias registradas globalmente: 4

Incidencias registradas: 2

Pérdida acumulada global: 142 €

Última incidencia: 2026-01-12

Nada más.

Nada de hoteles.

Nada de comentarios.

Nada de PII cruzada.

Ajuste técnico necesario antes de importar

Ahora debemos modificar ligeramente la función de stay para que también:

- Incremente `reservations_count`
- Recalcule `risk_band` aunque no haya incidencias

Te dejo versión correcta y definitiva.

create or replace function debacu_eval_upsert_guest_index_from_stay(

 p_identity_key text,

 p_activity_date date,

 p_is_completed boolean

)

returns void

```

language plpgsql
as $$
begin

insert into debacu_eval_import_guest_index (
    identity_key,
    stays_count,
    reservations_count,
    first_seen_date,
    last_seen_date
)
values (
    p_identity_key,
    case when p_is_completed then 1 else 0 end,
    1,
    p_activity_date,
    p_activity_date
)
on conflict (identity_key)
do update set
    reservations_count = debacu_eval_import_guest_index.reservations_count + 1,
    stays_count =
        debacu_eval_import_guest_index.stays_count +
        case when p_is_completed then 1 else 0 end,
    last_seen_date = greatest(
        debacu_eval_import_guest_index.last_seen_date,
        p_activity_date
    );

-- Recalcular risk_band si no tiene incidencias
update debacu_eval_import_guest_index
set risk_band =

```

```
case
  when incidents_count = 0 then 'LOW'
  when incidents_count = 1 and total_net_loss < 200 then 'MEDIUM'
  else 'HIGH'
end
where identity_key = p_identity_key;

end;

$$;
```

Regla operativa definitiva

Cuando importes:

- Reserva futura → p_is_completed = false
 - Estancia histórica → p_is_completed = true
 - Cambio a COMPLETED → llamas otra vez con true
-



Estado del sistema ahora

Ya tenemos:

- ✓ Historial negativo reconstruido
- ✓ Infraestructura import creada
- ✓ Índice global listo
- ✓ Modelo conceptual cerrado

Ahora sí toca lo importante

Construir la Edge Function:

debacu_eval_import_csv

Que hará:

1. Crear batch

2. Parsear CSV
3. Generar identity_key (HMAC)
4. Insertar guest_stays
5. Llamar función upsert índice
6. Consultar índice y devolver resultado por fila

la importación sera:

- A) Sincrónica (espera y devuelve resultado completo)
- B) Asíncrona (procesa en background y luego se consulta resultado)

Para CSV pequeños (hasta 3.000 filas), A es suficiente.

Para históricos grandes (50.000+), mejor B.

el sistema para soportar **ambos modos**, sin duplicar lógica y sin complicarlo innecesariamente.

La clave es esta:

Un único motor interno de procesamiento
Dos modos de ejecución (sync / async)

Diseño definitivo del Import Engine

1 Núcleo único de procesamiento

Creamos una función interna (no pública) que haga todo:

```
process_import_batch(batch_id)
```

Esa función:

1. Lee las filas del batch
2. Genera identity_key

3. Inserta en guest_stays (con dedupe)
4. Llama a:
 - debacu_eval_upsert_guest_index_from_stay
5. Consulta guest_index
6. Actualiza import_rows con:
 - match_status
 - risk_band_at_import

Eso es el motor real.

Modo A — Sincrónico (CSV pequeño)

Edge:

POST /debacu_eval_import_csv?mode=sync

Flujo:

1. Crea batch
2. Inserta filas
3. Llama a process_import_batch
4. Devuelve resultado inmediato

Uso ideal:

- Próximos check-in
- 50–3.000 filas
- Control diario

Modo B — Asíncrono (histórico grande)

Edge:

POST /debacu_eval_import_csv?mode=async

Flujo:

1. Crea batch
2. Inserta filas
3. Devuelve:

```
{ batch_id: X, status: "processing" }
```
4. Worker (otra función Edge o cron) procesa por chunks
5. Usuario consulta:

GET /debacu_eval_import_batch_status?id=X

Uso ideal:

- Carga inicial 2–3 años
 - 20.000 – 100.000 filas
 - PMS export completo
-



Arquitectura técnica limpia

CSV → import_batches
↓
import_rows
↓
process_import_batch
↓
guest_stays
↓
guest_index
Nada toca debacu_evaluations.



Ventaja clave

- No duplicas lógica.
 - El async usa el mismo motor.
 - El sync es simplemente “esperar a que termine”.
-



Decisión práctica importante

Para no complicar producción ahora:

Te recomiendo:

- 1 Implementar primero modo sync
- 2 Validar que todo funciona
- 3 Luego activar modo async

Porque el 90% de los hoteles usarán:

- Control diario
- CSV pequeños

El histórico grande lo harán una sola vez.

Pregunta crítica antes de escribir la Edge

¿Los CSV tendrán siempre:

- `checkin_date` obligatorio?
- `checkout_date` opcional?
- documento/email/teléfono al menos uno obligatorio?

Confírmame ese esquema mínimo y empezamos a escribir la Edge real.

Especificación oficial CSV – Debacu Import

● Campos obligatorios

- `checkin_date` → obligatorio
- Al menos uno de:
 - `document`
 - `email`
 - `phone`

● Campos opcionales

- `checkout_date`
- `full_name`
- `country` (si no viene → default ESP)

Reglas estrictas de validación

1 `checkin_date`

- Formato ISO: YYYY-MM-DD
 - Si no es válida → fila ERROR
 - No se inserta en `guest_stays`
 - Se guarda en `import_rows` con `error_code = INVALID_DATE`
-

Identificador obligatorio

Debe cumplirse:

document OR email OR phone

Si no → error_code = NO_IDENTIFIER

country

- Si viene → validar que sea código alpha-3
- Si no viene → asignar automáticamente:

ESP

Esto no afecta identity_key (solo sirve para estadística futura).

identity_key – Regla final definitiva

Orden jerárquico:

1. document_norm
2. email_norm
3. phone_digits

Nunca se combinan.

Siempre se usa el primero disponible.

Ejemplo:

DOC:11792515R

EMAIL:fercarboc@gmail.com

PHONE:34660082474

Luego:

HMAC_SHA256(DEBACU_GLOBAL_PEPPER, normalized_identifier)

Validación adicional recomendable (muy importante)

Para evitar basura masiva:

- email debe contener "@"
- phone mínimo 7 dígitos
- document mínimo 5 caracteres

Si no cumple → fila ERROR

No bloqueamos todo el batch.
Solo esa fila.

Flujo completo de importación (modo sync)

Por cada fila válida:

1. Normalizar datos
2. Generar identity_key
3. Insertar en:
 - debacu_eval_import_rows
 - debacu_eval_guest_stays (con dedupe)
4. Llamar:
`debacu_eval_upsert_guest_index_from_stay(identity_key, checkin_date, is_completed)`
5. Consultar índice global
6. Guardar en import_rows:
 - match_status
 - risk_band_at_import

match_status – Definición clara

Se calcula así:

Condición	Resultado
No existe en guest_index	NO_MATCH
risk_band LOW	MATCH_LOW
risk_band MEDIUM	MATCH_MEDIUM
risk_band HIGH	MATCH_HIGH

Resultado que devolverá la Edge (modo sync)

Por cada fila:

```
{  
  "full_name": "FERNANDO CARBONELL",  
  "checkin_date": "2026-03-01",  
  "match_status": "MATCH_LOW",  
  "risk_band": "LOW",  
  "stays_count": 5,  
}
```

```
"incidents_count": 0,  
"total_net_loss": 0  
}
```

Cuando importes un CSV histórico (estancias pasadas): Si checkin_date < today → COMPLETED

Lo cerramos así:

```
Si checkin_date < today → COMPLETED  
Si checkin_date >= today → PLANNED
```

Sin depender del usuario. Sin errores humanos.

Edge Function de Importación (modo sync)

Vamos a diseñarla limpia, sin romper nada, y reutilizable luego para async.

Nombre:

debacu_eval_import_csv

Modo inicial: sync

Lógica completa de la función

1 Entrada

Recibe:

```
{  
  "rows": [  
    {  
      "full_name": "...",  
      "document": "...",  
      "email": "...",  
      "phone": "...",  
      "country": "...",  
      "checkin_date": "...",  
      "checkout_date": "..."  
    }  
  ]  
}
```

2 Flujo interno

Por cada fila:

1. Validar checkin_date
2. Validar identificador mínimo
3. Normalizar:
 - document_norm
 - email_norm
 - phone_digits
4. Generar identity_key (HMAC con pepper)
5. Determinar stay_status
6. Insertar guest_stays (con dedupe)
7. Llamar:

```
debacu_eval_upsert_guest_index_from_stay(  
    identity_key,  
    checkin_date,  
    is_completed  
)
```

Consultar índice global

- Determinar match_status
- Insertar en import_rows

debacu_eval_import_csv (function a desplegar) para importacion

Qué hace esta versión

- ✓ Procesa fila por fila
- ✓ No bloquea por errores individuales
- ✓ Dedupe automático por índice único

- ✓ Actualiza índice global
- ✓ Devuelve resultado inmediato
- ✓ No toca debacu_evaluations
- ✓ Compatible con async futuro

Ahora debemos modificar la Edge de creación de incidencias para que llame a:

```
debacu_eval_upsert_guest_index_from_incident
```

Si no lo hacemos, el índice global se desincroniza cuando se cree una nueva incidencia.

Asegurarnos de que debacu_evaluations tiene identity_key

Si ya lo añadiste antes, perfecto.

Si no:

```
alter table debacu_evaluations  
add column if not exists identity_key text;
```

⚠ A partir de ahora, cuando crees nuevas incidencias desde Edge, debes generar identity_key allí también.

Funcion trigger

```
create or replace function debacu_eval_after_insert_incident()
```

```
returns trigger
```

```
language plpgsql
```

```
as $$
```

```
begin
```

```
-- Solo si tiene identity_key
```

```
if new.identity_key is not null then
```

```
    perform debacu_eval_upsert_guest_index_from_incident(
```

```
        new.identity_key,
```

```
        new.evaluation_date,
```

```
        new.economic_impact_gross,
```

```
new.economic_recovered,  
new.economic_net_loss  
);  
  
end if;  
  
return new;  
  
end;  
$$;  
  
crear gtrigger:  
  
drop trigger if exists debacu_eval_incident_trigger  
on debacu_evaluations;  
  
create trigger debacu_eval_incident_trigger  
after insert on debacu_evaluations  
for each row  
execute function debacu_eval_after_insert_incident();
```

Qué conseguimos con esto

- ✓ Cualquier incidencia nueva actualiza automáticamente el índice
- ✓ No depende del frontend
- ✓ No depende de Edge
- ✓ No rompe nada actual
- ✓ No modifica datos existentes

MUY IMPORTANTE

Ahora solo hay una condición:

Cada nueva incidencia debe tener `identity_key`.

Eso significa que en tu Edge de creación de incidencia debes:

1. Normalizar document/email/phone
2. Generar `identity_key` con HMAC
3. Insertar en `debacu_evaluations` incluyendo `identity_key`

Si no haces eso, el trigger no hará nada.

CONTINUIDAD PROYECTO – DEBACU EVALUATION (IMPORT + GLOBAL INDEX)

Estamos trabajando en Debacu Evaluation con la siguiente arquitectura ya implementada:

◇ Estado actual del sistema

1. `debacu_evaluations`

- Tabla principal en producción.
- Contiene incidencias.
- Tiene `identity_key`.
- Tiene trigger que actualiza índice global al insertar incidencia.

2. `debacu_eval_import_guest_index`

- Índice global anónimo.
- Campos:
 - `identity_key`
 - `stays_count`
 - `reservations_count`
 - `incidents_count`
 - `total_gross`
 - `total_recovered`
 - `total_net_loss`

- first_seen_date
 - last_seen_date
 - last_incident_date
 - risk_band
 - Alimentado por:
 - Backfill histórico desde debacu_evaluations
 - Trigger automático en nuevas incidencias
 - Importaciones de estancias
3. Tablas de import creadas:
- debacu_eval_import_batches
 - debacu_eval_import_rows
 - debacu_eval_guest_stays
4. Edge Function creada:
- debacu_eval_import_csv (modo sync)
 - Procesa fila por fila
 - Genera identity_key con HMAC usando DEBACU_GLOBAL_PEPPER
 - Inserta stays
 - Actualiza índice global
 - Devuelve resultado por fila
-

◇ Reglas ya decididas

- checkin_date obligatorio
- checkout_date opcional
- document/email/phone → al menos uno obligatorio
- country default ESP
- identity_key generado con HMAC SHA-256 + GLOBAL_PEPPER
- Si checkin_date < today → COMPLETED
- Si checkin_date >= today → PLANNED
- Mostrar en consulta global:
 - SIN INCIDENCIAS REGISTRADAS ☒

- Estancias registradas globalmente
 - Incidencias registradas
 - total_net_loss
 - risk_band
 - Nunca mostrar qué hotel
-

◇ **Objetivo del siguiente paso**

Construir el endpoint definitivo de:

Consulta Global de Huésped

Debe:

1. Recibir document/email/phone
2. Generar identity_key
3. Consultar debacu_eval_import_guest_index
4. Devolver:
 - exists
 - risk_band
 - stays_count
 - incidents_count
 - total_net_loss
 - last_seen_date
 - last_incident_date

Sin exponer PII ni hoteles.

Continuamos desde aquí.

Cuando abras el nuevo chat, pega esto y escribe:

Continuamos con la implementación del endpoint de consulta global.

Y seguimos exactamente desde ese punto.

Aquí tienes el “contrato de datos” completo (por lógica) dividido en 3 CSV: **Diario**, **Futuro** e **Histórico (entre fechas)**. Incluyo **campos MÍNIMOS / RECOMENDADOS / ÓPTIMOS** y para qué sirven (riesgo, auditoría, canal, fuga revenue, revenue intelligence).

Nota práctica: puedes exigir “máximo” pero debes permitir que falten opcionales. Lo que NO esté en el mínimo, el sistema sigue funcionando pero con menos precisión o menos módulos.

1) CSV DIARIO — “IN_HOUSE / ARRIVALS TODAY”

Objetivo: screening operativo del día (check-in hoy / in-house / check-out hoy) + alertas inmediatas.

MÍNIMOS (si falta alguno, el screening pierde sentido)

Identidad (mínimo 1 de estos):

- document_number
- email
- phone

Fechas:

- checkin_date
- checkout_date *(si no, al menos checkin)*

Nombre (para UI/soporte, no para identidad):

- first_name
- last_name

Referencia operativa (para dedupe y trazabilidad):

- reservation_ref *(si existe; si no, lo suplimos con identity+checkin)*

RECOMENDADOS (para “riesgo por canal” y mejor dedupe)

- channel *(Direct / Booking / Expedia / Agency...)*
- channel_group *(si ya lo traen; si no lo mapeas tú)*
- status *(IN_HOUSE / ARRIVAL / CHECKOUT / NO_SHOW / CANCELLED)*
- booking_created_at *(para pickup/anticipación)*

- adults, children
- room_number / room_id
- room_type
- rate_plan

ÓPTIMOS (para auditoría + reducir falsos positivos + revenue)

- document_type (*DNI/PASSPORT/ID*)
- document_country
- date_of_birth
- nationality
- guest_country
- city
- postal_code
- company_name (*si es corporate*)
- payment_type (*CARD/CASH/TRANSFER...*)

Revenue (si quieres fuga/ADR ya en diario):

- currency
 - total_amount (*total reserva*)
 - room_amount (*solo alojamiento*)
 - extras_amount
 - commission_amount (*si OTA*)
 - net_amount (*si lo calculan*)
-

2) CSV FUTURO — “FUTURE BOOKINGS (semana/mes/6 meses)”

Objetivo: screening preventivo de reservas futuras, y **delta** semana a semana aunque el cliente se repita.

MÍNIMOS

Identidad (mínimo 1):

- document_number (**ideal exigirlo aquí**)
- email
- phone

Fechas (clave):

- checkin_date
- checkout_date

Nombre:

- first_name
- last_name

Dedupe reserva:

- reservation_ref (*muy recomendable; si no, dedupe por identity+checkin*)

RECOMENDADOS (para Revenue Intelligence real)

- booking_created_at (*sin esto NO hay pickup serio*)
- status (*CONFIRMED / CANCELLED / OPTION / PENDING / NO_SHOW*)
- channel
- total_amount
- currency

Segmentación Revenue:

- market_segment (*LEISURE/CORPORATE/GROUP*)
- rate_plan
- board_code (*RO/BB/HB/FB*)

ÓPTIMOS (para forecast fino y riesgo por canal serio)

- commission_amount (*OTA*)
- net_amount (*si ya lo traen*)
- promo_code
- source_campaign (*si es directo con marketing*)

- deposit_amount
- cancellation_policy_code
- flexible_rate (*bool*)
- adults, children, rooms_count
- room_type
- unit_count (*si apartamentos*)

Identidad extra (reduce falsos positivos):

- document_type, document_country, date_of_birth, nationality
-

3) CSV HISTÓRICO — “PAST STAYS / RESERVATIONS (entre fechas)”

Objetivo: cargar 2–3 años para tener base de:

- incidentes propios del hotel (riesgo)
- producción por canal
- cancelaciones/no-show
- fuga de revenue
- futuros módulos de revenue intelligence (pickup histórico, ADR, etc.)

Aquí hay 2 variantes: **Histórico de Estancias** (lo que ocurrió) y **Histórico de Reservas** (pipeline pasado). Ideal: ambos. Si solo uno, prioriza “estancias” para impacto real.

3A) HISTÓRICO DE ESTANCIAS (recomendado como principal)

MÍNIMOS

- identidad: document_number o email o phone
- checkin_date
- checkout_date
- status (*CHECKED_OUT / NO_SHOW / CANCELLED / IN_HOUSE*)
- reservation_ref (*si existe*)
- channel

- first_name, last_name

RECOMENDADOS

- booking_created_at
- total_amount
- currency
- room_amount
- commission_amount (*si OTA*)
- net_amount (*si lo traen*)
- rate_plan
- room_type
- adults, children
- country / nationality

ÓPTIMOS

- cancelled_at (*si cancelada*)
 - no_show_flag
 - length_of_stay (*si lo traen; si no lo calculas*)
 - payment_status (*PAID/PARTIAL/UNPAID*)
 - invoice_id
 - company_name
 - notes (*limpiarlas; PII controlado*)
-

3B) HISTÓRICO DE RESERVAS (para pickup y forecast retrospectivo)

MÍNIMOS

- identidad: document_number o email o phone
- booking_created_at
- checkin_date
- checkout_date

- status
- reservation_ref
- channel
- total_amount, currency

RECOMENDADOS

- modified_at (*cambios; útil para pickup real*)
- cancelled_at
- rate_plan
- market_segment
- commission_amount, net_amount

ÓPTIMOS

- lead_time_days (*si no, lo calculas*)
 - promotion_code
 - source_campaign
-

Campos “comunes” transversales (siempre que puedas)

Estos son los que más valor te dan para cruzar y evitar líos:

Identidad / matching

- document_number
- document_type
- document_country
- email
- phone
- date_of_birth
- first_name
- last_name
- nationality

Reserva / estancia

- reservation_ref
- booking_created_at
- checkin_date
- checkout_date
- status
- channel (raw)
- channel_group (normalizado)
- market_segment
- rate_plan
- room_type
- adults, children, rooms_count

Revenue (si vendes fuga + revenue intelligence)

- currency
- total_amount
- room_amount
- extras_amount
- commission_amount
- net_amount
- payment_status
- deposit_amount

Auditoría / trazabilidad

- source_system (*nombre PMS / Channel Manager*)
 - exported_at (*cuando se generó el CSV*)
 - file_id (*si el sistema lo da*)
-

Reglas duras (para no vender humo)

- **Riesgo por canal:** sin channel solo puedes dar “riesgo huésped”, no por canal.
 - **Fuga de revenue:** sin total_amount (o net/room_amount) solo puedes estimar, y es discutible.
 - **Pickup / forecast:** sin booking_created_at no hay pickup serio.
-

Si quieres, el siguiente paso es que te lo deje como **plantillas CSV Debacu** (con headers exactos) para:

- debacu_inhouse_today.csv
- debacu_future_bookings.csv
- debacu_historical_stays.csv
- debacu_historical_bookings.csv

y además el esquema de “mapeo” para que en la UI el hotel asigne sus columnas a estas.

Perfecto. Vamos a dejar esto **cerrado y profesional**: te doy las **plantillas oficiales Debacu** con headers exactos.

Estas serán tus contratos de datos.

1 debacu_inhouse_today.csv

Objetivo:

Screening operativo del día + riesgo inmediato + trazabilidad.

reservation_ref, source_system, exported_at, booking_created_at, checkin_date, checkout_date, status, channel, market_segment, rate_plan, room_type, room_number, adults, children, currency, total_amount, room_amount, extras_amount, commission_amount, net_amount, payment_status, document_number, document_type, document_country, email, phone, date_of_birth, first_name, last_name, nationality, guest_country, city, postal_code, company_name

Campos realmente críticos:

- checkin_date
 - identidad (document_number o email o phone)
 - channel (si vendes riesgo por canal)
 - total_amount (si vendes fuga)
-

2 debacu_future_bookings.csv

Objetivo:

Screening preventivo + delta semanal + base Revenue Intelligence.

reservation_ref, source_system, exported_at, booking_created_at, checkin_date, checkout_date, status, channel, market_segment, rate_plan, room_type, adults, children, rooms_count, currency, total_amount, room_amount, extras_amount, commission_amount, net_amount, deposit_amount, cancellation_policy_code, document_number, document_type, document_country, email, phone, date_of_birth, first_name, last_name, nationality, guest_country

Obligatorio real si quieres Revenue serio:

- booking_created_at
 - checkin_date
 - channel
 - total_amount
 - identidad fuerte
-

3 debacu_historical_stays.csv

Objetivo:

Cargar 2-3 años de producción + base para riesgo propio + análisis por canal.

reservation_ref, source_system, exported_at, booking_created_at, checkin_date, checkout_date, status, channel, market_segment, rate_plan, room_type, adults, children, rooms_count, currency, total_amount, room_amount, extras_amount, commission_amount, net_amount, payment_status, document_number, document_type, document_country, email, phone, date_of_birth, first_name, last_name, nationality, guest_country, company_name

4 debacu_historical_bookings.csv (opcional pero potente)

Objetivo:

Pickup histórico + forecast comparativo + lead time analysis.

reservation_ref, source_system, exported_at, booking_created_at, modified_at, checkin_date, checkout_date, status, cancelled_at, channel, market_segment, rate_plan, room_type, adults, children, rooms_count, currency, total_amount, room_amount, extras_amount, commission_amount, net_amount, deposit_amount, cancellation_policy_code, document_number, email, phone, first_name, last_name

Reglas técnicas obligatorias

Fechas

Formato ISO:

YYYY-MM-DD
YYYY-MM-DD HH:MM:SS

Decimales

Siempre con punto:

1234.50

Currency

ISO 4217:

EUR, USD, GBP

Status estandarizado

CONFIRMED
IN_HOUSE
CHECKED_OUT
CANCELLED
NO_SHOW
OPTION
PENDING

Identity Strategy recomendada

Orden de prioridad para generar `identity_key`:

1. `document_number` + `document_country`
2. `document_number`
3. email (normalizado)
4. phone (digits only)
5. name + birth_date (fallback)

Si no hay al menos 3–4 campos de identidad → marcar `match_confidence` = LOW.

Lo que te permite esto comercialmente

Con estos CSV puedes vender:

Módulo	Requiere
Riesgo huésped	identidad
Riesgo por canal	channel

Módulo	Requiere
Fuga de revenue	total_amount + commission
Pickup	booking_created_at
Forecast comparativo	histórico + booking_created_at
ADR / RevPAR	total_amount + roomnights
Cancelación analysis	cancelled_at + status

⚠ Decisión estratégica (importante)

Si un hotel NO puede darte:

- channel
- total_amount
- booking_created_at

Entonces NO puede usar Revenue Intelligence completo.
Solo podrá usar “Riesgo y Auditoría”.

Eso debes dejarlo claro desde el onboarding.

Perfecto. Vamos a dejar esto bien diseñado desde el principio, sin parches futuros.

Te dejo el **modelo completo de base de datos para Supabase** listo para:

- perfiles de importación
- subidas de CSV
- dedupe de reservas (watchlist)
- ejecuciones de screening
- resultados con delta
- trazabilidad total

Todo alineado con tu arquitectura multi-tenant (org_id).

1 IMPORT PROFILES (mapeo de columnas por hotel)

```
create table public.import_profiles (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null,
  name text not null,
```

```

source_type text not null check (
  source_type in
('INHOUSE_TODAY','FUTURE_BOOKINGS','HISTORICAL_STAYS','HISTORICAL_BOOKINGS')
),
delimiter text default ',',
date_format text default 'YYYY-MM-DD',
decimal_separator text default '.',
encoding text default 'UTF-8',
identity_strategy text not null check (
  identity_strategy in
('DOCUMENT_STRONG','DOCUMENT','EMAIL','PHONE','NAME_DOB')
),
mapping jsonb not null, -- { "hotel_column": "debaçu_field" }
disabled_fields text[],
created_at timestampz default now(),
updated_at timestampz default now()
);
create index on public.import_profiles(org_id);

```

2 IMPORT JOBS (cada CSV subido)

```

create table public.import_jobs (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null,
  user_id uuid not null,
  profile_id uuid references public.import_profiles(id),
  file_path text not null,
  file_hash text,
  run_type text not null check (
    run_type in
('INHOUSE_TODAY','FUTURE_BOOKINGS','HISTORICAL_STAYS','HISTORICAL_BOOKINGS')
),
total_rows integer default 0,
valid_rows integer default 0,
invalid_rows integer default 0,
status text not null default 'UPLOADED' check (
  status in ('UPLOADED','VALIDATED','COMMITTED','FAILED')
),
summary jsonb,
created_at timestampz default now(),
updated_at timestampz default now()
);
create index on public.import_jobs(org_id);
create index on public.import_jobs(profile_id);

```

3 WATCHLIST RESERVATIONS (dedupe operativo)

Esta es la clave de tu sistema preventivo.

```

create table public.watchlist_reservations (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null,
  reservation_ref text,
  identity_key text not null,
  checkin_date date not null,

```

```

checkout_date date,
channel text,
total_amount numeric default 0,
currency text,
status text,
first_seen_at timestamptz default now(),
last_seen_at timestamptz default now(),
created_at timestamptz default now(),
updated_at timestamptz default now()
);

```

DEDUPE

Si hay reservation_ref:

```

create unique index uniq_watchlist_res_ref
on public.watchlist_reservations(org_id, reservation_ref)
where reservation_ref is not null;

```

Fallback si no hay referencia:

```

create unique index uniq_watchlist_identity_date
on public.watchlist_reservations(org_id, identity_key, checkin_date);

```

4 SCREENING RUNS (cada ejecución analítica)

```

create table public.screening_runs (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null,
  import_job_id uuid references public.import_jobs(id),
  run_type text not null check (
    run_type in
('INHOUSE_TODAY', 'FUTURE_BOOKINGS', 'HISTORICAL_STAYS', 'HISTORICAL_BOOKINGS')
),
total_analyzed integer default 0,
high_count integer default 0,
medium_count integer default 0,
low_count integer default 0,
created_at timestamptz default now()
);
create index on public.screening_runs(org_id);

```

5 SCREENING RESULTS (resultado por cliente por run)

Aquí está el oro comercial.

```

create table public.screening_results (
  id uuid primary key default gen_random_uuid(),
  run_id uuid not null references public.screening_runs(id) on delete cascade,
  org_id uuid not null,
  identity_key text not null,
  checkin_date date,
  risk_band text not null,
  incidents_count integer,

```

```
total_net_loss numeric,  
last_incident_date date,  
days_since_last integer,  
delta_incidents_count integer default 0,  
delta_total_net_loss numeric default 0,  
match_confidence text check (  
    match_confidence in ('HIGH','MEDIUM','LOW')  
),  
match_basis text, -- DOCUMENT / EMAIL / PHONE / NAME_DOB  
computed_at timestampz default now()  
);  
create index on public.screening_results(org_id);  
create index on public.screening_results(run_id);  
create index on public.screening_results(identity_key);  
Clave única lógica:  
  
create unique index uniq_screening_per_run  
on public.screening_results(run_id, identity_key, checkin_date);
```

Flujo completo

- 1 Hotel sube CSV
→ import_jobs creado
 - 2 Edge Function valida y parsea
→ genera/actualiza watchlist_reservations (dedupe correcto)
 - 3 Se crea screening_runs
 - 4 Para cada fila:
 - Lee debacu_eval_import_guest_index
 - Calcula riesgo actual
 - Busca resultado del run anterior
 - Calcula delta
 - Inserta en screening_results
 - 5 UI muestra:
 - semáforo
 - cambios desde la última vez
 - detalle incidentes
-

Multi-tenant

En todas las consultas siempre:

```
where org_id = current_org_id
```

Y aplica RLS como ya haces en Debacu Evaluation.



Esto ya te permite vender

- Screening automático semanal
 - Alertas de riesgo antes del check-in
 - Evolución del huésped
 - Riesgo por canal
 - Fuga de revenue
 - Pickup futuro
 - Forecast comparativo (cuando conectes revenue)
-

Lo siguiente (estratégico)

Ahora tienes que decidir:

1. ¿El screening se ejecuta inmediatamente tras cada import?
2. ¿O lo programamos también nocturno por si hay nuevas incidencias globales?

Mi recomendación seria:

- Ejecutar en cada import
- Y además job nocturno para recalcular reservas futuras activas

funcion **import_validate_commit**

Lo que hace exactamente (importante para que no haya sorpresas)

Validación

- Usa `profile.mapping` para transformar headers del hotel → fields Debacu.
- Exige mínimos según `run_type` (y respeta `disabled_fields`).

- Calcula `identity_key` con fallback:
 - documento → email → phone → name+dob (si existe)
- Si no hay identidad, esa fila se marca inválida.

Commit

- Crea `import_jobs` y `screening_runs`
- Upsert en `watchlist_reservations` con dedupe:
 - si hay `reservation_ref: org_id, reservation_ref`
 - si no: `org_id, identity_key, checkin_date`
- Lee el riesgo **de tu índice agregado**: `debacu_eval_import_guest_index`
- Inserta `screening_results` con:
 - snapshot actual
 - `delta_*` contra run anterior

Siguiente paso: algoritmo exacto del delta (ya lo tienes, pero lo afinamos)

Ahora mismo el delta es:

- `delta_incidents_count = current.incidents_count - prev.incidents_count`
- `delta_total_net_loss = current.total_net_loss - prev.total_net_loss`

Y el “match” del previo es:

- por `(identity_key + checkin_date)` si tienes checkin
- si no, por `identity_key` (en el helper `getPrevResult` lo dejé listo para extender)

Lo siguiente (y es lo que tú quieres de verdad) es mejorar a:

- delta por **nuevos incidentes** (no solo diferencias numéricas)
- y delta aunque cambie el check-in o se repita en otro CSV (`identity_key` puro)

por esto por `identity_key`

el **delta** será siempre por `identity_key`, independientemente del `checkin_date`.

Eso es lo correcto a nivel de producto, porque:

- El riesgo es del huésped, no de la reserva.
- Si el cliente empeora en otro hotel esta semana, aunque sea para otra fecha, el hotel debe verlo.
- Evitas falsos negativos si cambia la fecha o se cancela y vuelve a reservar.

Ahora vamos a hacerlo **bien y eficiente**, no con 1 query por fila.

Qué significa ahora el delta

Ejemplo real:

Semana pasada:

```
incidents_count = 1  
total_net_loss = 50
```

Hoy:

```
incidents_count = 3  
total_net_loss = 120
```

Resultado:

```
delta_incidents = 2  
delta_loss = 70
```

Aunque:

- haya cambiado el check-in
- esté ahora en otra reserva
- se haya cancelado la anterior

El riesgo del huésped subió. Eso es lo que importa.

🤔 ¿Qué pasa si el cliente no existía en el run anterior?

Entonces:

```
delta_incidents = current_incidents  
delta_loss = current_loss
```

Es correcto:

Es la primera vez que el hotel lo analiza.



Nivel profesional (muy recomendable)

Añade además:

```
delta_risk_changed: boolean
```

Comparando banda actual vs banda previa:

```
const prevBand = prevBandMap.get(identity_key) || null;  
const risk_changed = prevBand && prevBand !== risk_band;
```

Eso en UI vende muchísimo:

“Este huésped ha cambiado de MEDIUM → HIGH desde la última revisión”



Resumen

Ahora tu sistema:

- No duplica reservas
 - Recalcula siempre contra índice global
 - Detecta cambios reales entre ejecuciones
 - Escala bien (1 query para prev run)
 - Es coherente con tu modelo de riesgo
-

Si quieres subir un nivel más (muy interesante):

Podemos hacer que el delta detecte:

- ● nuevos incidentes específicos (IDs nuevos)
- ● nuevas pérdidas económicas aunque el count no cambie
- ● mejora por antigüedad (degradación automática)

Vamos a diseñar la **UI completa del módulo Screening / Import CSV** como si fuera un SaaS premium (alineado con Debacu Evaluation).

Te lo estructuro en:

1. Flujo completo UX
 2. Pantalla Subida CSV
 3. Pantalla Resultados del Run
 4. Panel de Alertas
 5. Detalle por identity_key
 6. Consideraciones técnicas reales (sin humo)
-

Flujo UX Completo

El hotel verá:

Menú lateral

Screening

- Subir CSV
- Resultados
- Alertas
- Perfiles de Importación

Flujo normal:

1. Subir CSV
2. Validar
3. Confirmar commit
4. Ver resultados del run
5. Revisar alertas
6. Entrar al detalle del cliente

Todo automático. Nada manual uno a uno.

2 Pantalla: “Subir CSV”

Ruta:

/screening/import

Layout

Header

Título grande:

Screening de Clientes

Sube un CSV para analizar riesgo e incidencias

Card 1 – Perfil de Importación

Selector:

Perfil:

[FUTURE_BOOKINGS ▼]

Botón:

Editar Mapping

Mostrar resumen del perfil:

Estrategia identidad: DOCUMENT_STRONG

Campos obligatorios: checkin_date, first_name, last_name

Campos deshabilitados: email

Card 2 – Subida

Drag & Drop area grande:

Arrastra aquí tu CSV
0
[Seleccionar archivo]
Debajo:

Formato esperado: UTF-8, delimitador coma

Card 3 – Validación (tras upload)

Después de dry_run:

Mostrar:

Total filas: 5.000
Válidas: 4.872
Errores: 128
Botones:

[Descargar errores]
[Confirmar Importación]
Si errores > 20%, mostrar warning rojo.



3 Pantalla: Resultados del Run

Ruta:

/screening/runs/:runId

Header

Screening – FUTURE_BOOKINGS
Fecha ejecución: 25 Feb 2026
Analizados: 4.872

Métricas superiores (4 cards)

HIGH MEDIUM LOW Alertas nuevas

Ejemplo:

HIGH: 42
MEDIUM: 312
LOW: 4.518
Alertas nuevas: 7
Color coding claro.

Tabla principal

Columnas:

- Checkin
- Identity Key (oculto → mostrar hash corto)
- Risk Band (chip)
- Δ Incidentes
- Δ Pérdida €
- Incidentes Totales
- Pérdida Total €
- Días desde último
- Confianza match
- Acción

Filtros arriba:

[Risk: ALL ▼]
[Solo con alertas ✓]
[Solo cambios riesgo ✓]
[Buscar identidad...]

Ejemplo fila

| 12/08/2026 | HIGH  | +1 | +120€ | 3 | 450€ | 2 días | HIGH | [Ver detalle] |

Si `risk_band_changed = true`, mostrar icono 



4 Pantalla: Alertas

Ruta:

/screening/alerts

Tabla:

| Fecha | Tipo | Identity | Mensaje | Estado |

Tipos:

- HIGH_RISK
- NEW_INCIDENT
- RISK_CHANGED

- DELTA_CHANGE

Botón:

[Marcar como revisada]
Esto actualiza `resolved_at`.

Filtros:

[Solo abiertas]
[Por tipo ▼]
[Por fecha ▼]



5 Detalle por identity_key

Ruta:

`/screening/identity/:identity_key`

Layout tipo dashboard

Columna izquierda

Card 1:

Riesgo actual: HIGH
Incidentes totales: 3
Pérdida acumulada: 450€
Último incidente: 24/02/2026

Card 2:

Histórico por run:
Run 1 → MEDIUM
Run 2 → HIGH
Run 3 → HIGH (+1 incidente)

Columna derecha

Timeline:

24/02/2026 → Incidente 120€
03/01/2026 → Incidente 80€
22/10/2025 → Incidente 250€

Si en el futuro añades tabla incidents real, aquí se conecta.



6 Diseño Visual (alineado con Debacu)

- Fondo azul oscuro
- Cards gris claro
- Chips:

- HIGH → rojo
- MEDIUM → naranja
- LOW → verde
- Iconos lucide-react
- Tipografía compacta
- Tabla densa tipo SaaS financiero

Nada infantil.

7 Comportamiento importante

Si suben el mismo CSV la semana siguiente

El sistema:

- No duplica watchlist
- Calcula delta contra run anterior
- Puede generar nuevas alertas si cambió riesgo

UI debe mostrar:

Comparado contra ejecución 18 Feb 2026

8 Rendimiento UI

Para 50k filas:

- Paginación server-side
 - Filtro por risk_band en query
 - No cargar todo en memoria
 - Debounce en search
-

9 Estructura de Componentes React (sugerida)

```
ScreeningLayout
├── ImportPage
├── RunsListPage
└── RunDetailPage
```

00 Lo que realmente vende esto

No es “subir CSV”.

Es:

- Control preventivo antes de llegada
- Seguimiento riesgo acumulado
- Detección de cambio semanal
- Alertas automáticas
- Base para Revenue Intelligence

CONTINUIDAD – MÓDULO SCREENING CSV (DEBACU EVALUATION)

Estamos trabajando en el módulo **Screening por CSV** dentro de Debacu Evaluation.

Ya tenemos implementada la Edge Function `import_validate_commit` con:

- `identity_key` por HMAC (DOCUMENT_STRONG / EMAIL / etc.)
- `screening_runs`
- `screening_results`
- `prev_risk_band`
- `risk_band_changed`
- `delta_incidents_count`
- `delta_total_net_loss`
- `screening_alerts`
- degradación por antigüedad (decay factor)
- precarga de `debacu_eval_import_guest_index` para rendimiento 50k filas
- bulk upsert `watchlist_reservations`
- detección de run anterior por `identity_key`

Ahora queremos:

1. Confirmar que todas las tablas necesarias existen correctamente en la BD.
2. Revisar estructura exacta de:
 - `screening_runs`

- screening_results
- screening_alerts
- import_jobs
- import_profiles
- watchlist_reservations

3. Verificar índices necesarios para rendimiento.
4. Si falta algo, generar SQL de ajuste.
5. Luego diseñar la UI real conectada a estas tablas.

Por favor:

- Dime qué consultas SQL ejecutar (`\d+ tabla` o `SELECT` columnas)
- Analiza si falta alguna columna clave
- Sugiere índices necesarios
- Valida que el sistema esté preparado para 50k filas por run